Guia de gravação da FIAP Store

Ronaldo Madukaife John Attamah | RM 564630

Roteiro de 20 minutos para apresentar a arquitetura e demonstrar o ambiente. A gravação é material adicional: o enunciado recebido exige demonstração ao vivo e não aceita vídeo como substituição.

Antes de gravar

Inicie o Learner Lab, renove as credenciais locais e mantenha a sessão ativa. Abra o terminal na raiz do projeto. Não mostre AWS Details, arquivos de credenciais, valores de Secrets ou senhas no vídeo.

Abra a loja em HTTPS e o repositório https://github.com/MaduKa01/fiap-store. Acesse o Argo CD por port-forward. O certificado do laboratório é autoassinado; resolva o aviso manualmente antes de iniciar a gravação.

```
export KUBECONFIG="$PWD/.kubeconfig"
export PATH="$PWD/.runtime/bin:$PATH"
export AWS_PROFILE=default AWS_REGION=us-east-1
aws sts get-caller-identity --query Account --output text
kubectl -n argocd port-forward svc/argocd-server 8081:443
```

Fala sugerida Esta é a FIAP Store implantada no Amazon EKS. Vou apresentar a arquitetura, mostrar as evidências e demonstrar como a plataforma reage aos testes.

Minutos 0 a 2 Apresente a arquitetura

Mostre o diagrama e a loja. Explique: navegador → balanceador AWS → Traefik → Ingress → API e front. A API grava no Postgres e coloca pedidos na fila Redis; o worker processa a fila.

Mostre quatro nós distribuídos em três zonas. A API mantém pelo menos três réplicas; o worker pode ficar em zero. O banco é uma instância única com EBS em uma zona, portanto sua recriação pode interromper temporariamente a API.

```
kubectl get nodes -L topology.kubernetes.io/zone
kubectl -n store-prod get deploy,hpa,scaledobject
kubectl -n store-data get sts,svc,pvc
```

Fala sugerida As réplicas da API são distribuídas entre zonas. O banco usa um volume persistente, mas esta configuração não oferece failover automático do banco entre zonas.

Minutos 2 a 7 Execute a avaliação

Execute o script do professor inteiro, sem cortes. Mostre a pontuação real da sessão. O resultado salvo de 05/11/2026 foi 80 de 80 pontos automáticos; isso não substitui a nova execução nem os 20 pontos avaliados manualmente.

Mostre o Argo CD em Synced e Healthy. Abra o resumo Trivy: as imagens avaliadas não tinham CRITICAL; há achados HIGH na API e no worker. Explique a correção do front 1.0 para 1.1, sem alterar HTML ou JavaScript.

```
bash scripts/07-validar.sh
kubectl -n argocd get application fiap-store
cat avaliacao/trivy/resumo.json
```

Fala sugerida O script verifica 80 pontos. Os relatórios de vulnerabilidades e a correção aplicada ao front estão versionados no repositório. Não estamos afirmando que as imagens estão livres de todo risco.

Minutos 7 a 16 Demonstre três desafios

Sugestão para o vídeo: self-healing da API, escala do worker com 50 pedidos e correção de drift pelo Argo CD. Para a apresentação ao vivo, prepare também os outros sete desafios, porque o professor pode sortear qualquer um.

Antes de cada ação, diga o estado inicial e o resultado esperado. Execute, mostre a mudança e explique qual controlador a produziu. Cada integrante deve conduzir pelo menos um desafio na avaliação ao vivo.

Fala sugerida Vou provocar uma falha controlada e observar o mecanismo de recuperação. O resultado precisa aparecer no cluster; apenas mostrar o manifesto não comprova o comportamento.

Minutos 16 a 20 Explique segurança e limites

Mostre que o front não alcança o banco e que a API alcança. Explique a ServiceAccount da API e o grupo fiap:devs. Mostre o PVC e as revisões da API.

Feche com o repositório, a declaração de uso de IA e o procedimento de destruição. Não execute a destruição durante o vídeo ou antes da demonstração.

```
kubectl -n store-prod get rs -l app=store-api
kubectl -n store-prod get networkpolicy
kubectl -n store-data get networkpolicy
kubectl -n store-prod get pdb
kubectl -n store-data get pvc
```

Fala sugerida A plataforma aplica menor privilégio, isolamento de rede e reconciliação pelo Git. O banco único e a fila efêmera são limitações conhecidas. A remoção do ambiente fica documentada para evitar consumo de créditos após o uso.

Três demonstrações para gravar

Self healing da API

Observe um novo Pod surgir e voltar a Ready. O Deployment recompõe a réplica. Depois: kubectl -n store-prod rollout status deploy/store-api.

```
kubectl -n store-prod delete pod $(kubectl -n store-prod get pod -l app=store-api -o jsonpath='{.items[0].metadata.name}')
```

Escala a zero

Com worker em zero, gere 50 pedidos pelo painel da loja. Mostre o worker subir, os pedidos serem processados e o retorno a zero. Pare a observação com Control+C.

```
kubectl -n store-prod get deploy store-worker -w
```

Correção de drift

Mostre o Service ser recriado pelo Argo CD. Observe com: kubectl -n store-prod get svc store-front -w. Pode existir breve indisponibilidade enquanto o Service está ausente.

```
kubectl -n store-prod delete service store-front
```

Prepare os outros desafios

- Drenagem: escolher nó sem store-db-0, executar drain e sempre finalizar com uncordon. Não usar force nem drenar o nó do banco.
- Rollback: alterar apenas a tag da API no manifesto, fazer commit e push; aguardar o Argo. Retornar de 2.0 para 1.0 e depois restaurar 2.0 pelo Git.
- Persistência: comparar pedidos e PVC antes e depois de recriar somente o Pod store-db-0. Não excluir StatefulSet, PVC ou volume.
- Pod Security: tentar criar Pod privilegiado em store-prod e mostrar a recusa por restricted.
- RBAC: fiap:devs pode listar Pods e ler logs; não pode apagar Pods, ler Secrets nem executar exec.
- Rede: conexão TCP do front ao banco falha; a mesma conexão pela API funciona.
- Relatório: criar Job a partir do CronJob store-report, aguardar Complete e mostrar o relatório novo na página.

Ao terminar

Confirme que o nó drenado está habilitado, a API está em 2.0, o front em 1.1 e o Argo CD em Synced e Healthy. Execute a avaliação novamente se algum teste alterou o ambiente. Não exponha segredos nas capturas. A data da apresentação deve ter sessão ativa e credenciais válidas.